Lenguaje de Programación

¿Que es un lenguaje de programación?

Es un conjunto de reglas sintácticas, semánticas y símbolos formales que permite a un ser humano escribir instrucciones estructuradas para que una computadora las interprete, procese y ejecute.

Elementos esenciales de un lenguaje de programación:

Sintáxis: Conjunto de reglas gramáticales que dictan como deben escribirse las palabras, simbolos y estructuras para que el código sea valido.

Semántica: Es el significado o la acción lógica que realiza cada instrucción escrito.

Vocabulario / Palabras Reservadas: Palabras claves con significado especial para el lenguaje que no pueden usarse para otros fines. Ejm: print, cout, while, retunrd.

Programación: Es el proceso de diseñar, escribir, probar y mantener código, puente para dar instrucciones a una computadora, permitiendole ejecutar tareas específicas o resolver problemas de forma automatizada.

Algoritmo: Es una secuencia finita ordenado y lógico de pasos o instrucciones presisas que deben seguirse para realizar una tarea o resolver un problema concreto.

Lenguaje de alto nivel: Es un lenguaje diseñado para ser leído y escrito facilmente para los seres humanos. Utiliza palabras en ingles y estructuras lógicas cercanas de lenguaje natuo, independizando al programador de la arquitectura interno de hadword. Ejm. C, c++, Piyton, JavaScript, C#.

Lenguaje de bajo nivel: Es un lenguaje que opera con mínima o nula abstracción del hadword, diseñado para comunicarse e interactuar directamente con la memoria y el procesador de la computadora. Ejm: lenguaje máquina y lenguaje asemberd.

LayConsa

Entornos de desarrollos integrados (IDE)

Es una aplicación de software que reune todas las herramientas necesarias para que un desarrollador escriba y pruebe código en un solo lugar.

Componentes principales de un IDE:

Editor de código fuente: Incluye resaltado de sintáxis, autocompletado y detección de errores en tiempo real.

Compiladores / Intérpretes integrado: Permite ejecutar el programa con un solo clip.

Depurador (Debugger): Herramientas para inspeccionar la memoria, avanzar linea por línea y rastrear errores.

Gestor de proyectos: Organiza multiples archivos, bibliotecas y configuraciones del programa

Variables, Constantes y tipos de datos

Variable: Una variable es un espacio reservado en la memoria RAM del ordenador que se utiliza para almacenar un valor que puede cambiar a lo largo de la ejecución del programa. Se utilizan para evitar errores y presentar valores fijos del dominio.

Ejemplo:

- Num1 = 5 num2 = 10 base = 10
- Convertir grados celsius a grados Fahrenheit.

Constante: Una constante es un valor almacenado en memoria que no puede cambiar ni ser modificado una vez que ha sido definido durante la ejecución del programa. Se utilizan para evitar errores y representar valores fijos del dominio.

Ejemplo = 3.141615

/ /

Tipos de datos:

Categoría	Tipo de dato	Tamaño Habitual	Rango (aproximado)
Entero	int	4 bytes	-2147483648 a 2147483647
	short	2 bytes	-32768 a 32767
	long	4 u 8 bytes	-2147483648 a 2147483647 (4 bytes); -9223372036854775808 a 9223372036854775807
	long long	8 bytes	-9223372036854775808 a 9223372036854775807
Punto flotante	float	4 bytes	$\pm 3,4 \times 10^{-38}$ a $\pm 3,4 \times 10^{38}$
	double	8 bytes	$\pm 1,7 \times 10^{-308}$ a $\pm 1,7 \times 10^{308}$
Caracteres	char	1 byte	-128 a 127
	wchar_t	2 o 4 bytes	0 a 65535 / a 4294967295 depende del sistema
Lógicos	bool	1 byte (típico)	0, 1 (true o false)

DFD => (Diagrama de Flujo de Datos)

Es una representación gráfica que muestra cómo fluye la información a través de un sistema o proceso.

Símbolo	Nombre	Función
Cuadro / Rectángulo	Entidad externa	Representa la fuente u origen de los datos
Círculo / Ovalo	Proceso	Operación o función que transforma los datos.
Líneas horizontales / Rectángulo abierto	Almacén de datos	Lugar donde se guardan los datos temporal o permanentemente
Flecha	Flujo de datos	Muestra la dirección en la que viaja la información

LayConsa

PSelnt:

Es una herramienta educativa de software libre diseñada para ayudar a estudiantes a dar sus primeros pasos en la programación y el desarrollo de la lógica algorítmica.

Utiliza pseudocódigos en español, lo que permite centrarse en los conceptos fundamentales de la programación (variables, condicionales, bucles y funciones). sin la complicación de la sintaxis escrita de un lenguaje real (como C++, Java o Python).

5.1 Ejercicios:

Conversión de tiempo (segundos a minutos / horas). Leer un valor en segundos e imprimir a cuantos minutos y horas equivale.

Cálculo del IGV de un producto: Solicita el precio base de un producto y cálculo el monto del IGV (18%) junto con el precio total final.

Área y perímetro de un rectangulo: Pide la longitud de la base y la altura de un rectangulo para imprimir su área y perímetro.

Operadores:

Operadores aritméticos: +, -, *, /, %, ++, --.

Operadores relacionales: ==, !=, >, <, >=, <=.

Operadores lógicos &&, ||, ! (en PSelnt: AND, O, NOT)

Estructuras Condicionales:

7.1 If-else: Las estructuras if-else son sentencias de control condicional que permiten tomar decisiones en el codigo ejecutandose bloques de instrucción únicamente cuando se cumple una condición.

1) Estructura if (condicional simple): Evalúa una condición. Si la condición resulta verdadera (true), se ejecuta dentro del bloque. Si es falsa (false), el programa omite ese bloque y continuó con la siguiente instrucción.

2) Estructura if - else (condicional doble): Permite definir un camino alternativo. Si la condición resulta verdadera (true) se ejecuta el primer bloque. Si es falsa (false) se ejecutan obligatoriamente el bloque dentro del else.

Ejemplo: par o impar

```
if (numero % 2 == 0)
    // par
else {
    // impar
```

Otro caso con if/else if: clasificar un número como positivo, negativo o nulo.

7.2 Switch - case

En c++, switch / case es una estructura de control de selección multiple que evalúa el valor de una variable y ejecuta un bloque de código según la coincidencia con una lista de casos enteros o caracteres.

Ejemplo:

```
switch (variable) {
    case var1:
        // ejecuta un código
        break;
    case var 2:
        // ejecuta un código
        break;
    default:
        // código si no coincide con ninguno previo case
        break;
}
```

LajConsa

Ejercicios

Crear un codigo c++ para la clasificación de glucosa en sangre para la detección de hipoglucemia, diabetes o prediabetes.

Completar hasta del 1 al 10 000 en el código de números a letras.

Estructuras Repetitivas

También llamada bucle, ciclo o iteración. Es un bloque de código que permite ejecutar una o varias instrucciones de forma reusriente mientras se cumpla una determinada condición lógica.

Su propósito principal es automatizar tareas repetitivas, evitar la duplicidad de códigos y procesar colecciones de datos (como listas o arreglos) de manera eficiente.

```
for (inicio; final; incrementa)
for (i=1; i<=10; i+1)
```

Iteración: Una iteración es cada una de las repeticiones o ejecuciones individuales de un bucle. Si un bucle se ejecuta 5 veces, se dice que se realiza 5 iteraciones.

Contador: Es una variable entera cuyo valor se incrementa o decrementa en una cantidad constante (generalmente en 1) en cada iteración. Se utiliza para registrar la cantidad de veces que ocurre un evento o se repite un ciclo.

```
contador = contador + 1;
contadorPar = contadorPar + 1;
```

Acumulador: Es una variable que almacena la suma o multiplicación sucesiva de valores variables en cada iteración. Se utiliza para obtener totales, promedios o sumatorias acumuladas.

suma = suma + i;

suma += i;

8.1 Control de flujo iterativo

C++ proporciona sentencias especiales para alterar la ejecución normal de una estructura repetitiva.

break: Interrumpe y sale inmediatamente del bucle, ignorando las iteraciones restantes.

Continue: Salta el resto de las instrucciones de la iteración actual y pasa directamente a la siguiente iteración (evaluando de nuevo la condición de incremento).

8.2 Anidamiento básico de bucles

Tabla de multiplicación

1	x	1	=	1					2	x	1	=	2	...					
1	x	2	=	2					2	x	2	=	4	...					
⋮									⋮										
1	x	10	=	10					2	x	10	=	20		10	x	10	=	100

```
for (i = 1; i <= 10; i++)
    for (j = 1; j <= 10; j++)
        cout << i << "x" << j << "=" << i*j << endl;
```

8.3 Estructura repetitiva while (mientras)

Es un bucle o ciclo de control que ejecuta un bloque de código de forma iterativa mientras se cumpla una condición lógica determinada. A diferencia de otros bucles como for, el bucle while se utiliza principalmente cuando no se conoce como exactitud el número exacto de iteraciones que se van a realizar antes de iniciar la ejecución.

LaConsa

Funcionamiento básico

1) Evaluación: Antes de ejecutar el código se evalua la condición lógica.

2) Ejecución: Si la condición es verdadera (true), se ejecuta el cuerpo del bucle.

3) Repetición: Al terminar el bloque, se vuelve a evaluar la condición. El proceso continua hasta que la condición pase a ser falsa (false).

4) Finalización: Si la condición es falsa desde el inicio, el bloque dentro del bucle no se ejecutará ni una sola vez.

Estructuras Repetitivas do-while (hacer-mientras)

Es un bucle o ciclo de control que ejecuta un bloque de código al menos una vez y luego continua repitiendolo mientras se cumpla una condición lógica determinada. A diferencia del bucle while tradicional, la condición de evaluación se encuentra al final del código.

Funcionamiento básico

1) Ejecución obligatoria: El programa entra al ciclo y ejecuta las instrucciones del bloque directamente, sin comprobar ninguna condición previa.

2) Evaluación posterior: Al llegar al final del bloque, se evalúa la condición lógica expresada en while.

3) Repetición: Si la condición es verdadera (true), el flujo regresa al inicio del bloque para repetir el proceso.

4) Finalización: Si la condición es falsa (false), el ciclo termina y el programa continua con la instrucción siguiente.

Ejercicios:

- Leer n números enteros, obtener la suma total y el promedio de los numeros ingresados.

- Crear la tabla de multiplicación del 7.

Arreglo unidimensional (vector)

Un arreglo unidimencional es una colección lineal de datos que se dirige, ningun de forma secuencial, uno al lado del otro en la memoria del computador. Se puede visualizar como una sola fila con multiples casillas.

Ejemplo

tipo de dato nombre
int notas [5]

indices	0	1	2	3	4
	2	14	15	16	20

Arreglo bidimensional (matriz)

Un arreglo bidimensional es una estructura organizada en una tabla de dos dimensiones: filas y columnas. Se puede imaginar como una cuadricula o una hoja de calculo, donde la interseccion de una fila y una columna determina una posición o casilla especifica.

Ejemplo

int matriz [2][3]

	0	1	2
0	14	15	16
1	6	13	70